Database Schema

Student	Samson Odongo
ID	SSAMODO111111
Stack	PostgreSQL 16 + Entity Framework Core 8
Total tables	8 (6 application tables + 4 ASP.NET Identity tables)
Date	April 2026

■ PK — Primary Key	■ FK — Foreign Key (purple background)	✗ NOT NULL ✓ NULLABLE	Teal highlight = UNIQUE constraint

Table of Contents

1.	**organizations**	Orgs, plan tier, meeting usage counters
2.	**users**	App users — linked to ASP.NET Identity
3.	**meetings**	Meeting sessions — core entity of the app
4.	**transcripts**	Whisper ASR output per meeting
5.	**minutes**	AI-generated draft and approved meeting minutes
6.	**minutes_versions**	Edit history — one row per save
7.	**email_recipients**	Per-org email distribution list
8.	**email_delivery_log**	SendGrid delivery tracking per send event
	Identity tables	AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims
	All relationships	Full ER summary
	Migration commands	EF Core CLI commands to apply schema

organizations — One row per company or team using the app

The root entity. Every piece of data in the system belongs to an organization. Plan tier and usage counters live here — enforced on every request by the [RequiresPlan] attribute and MeetingService.

Column	Type	Null	Default	Description / Constraint
id	UUID	✗	gen_random_uuid()	Primary key — used in every FK across all tables
name	VARCHAR(255)	✗	—	Organization display name shown in the UI
plan	SMALLINT	✗	0	0=Free 1=Pro 2=Enterprise — stored as int, mapped to PlanTier enum in C#
meetings_used_this_month	INT	✗	0	Incremented on each new meeting. Reset to 0 on plan_renews_at. Free plan cap = 5
plan_renews_at	TIMESTAMPTZ	✗	NOW()+1 month	When meetings_used_this_month resets. Set to first day of next month on register
created_at	TIMESTAMPTZ	✗	NOW()	Org creation timestamp (UTC)
updated_at	TIMESTAMPTZ	✗	NOW()	Last updated. Maintained by EF Core ValueGeneratedOnAddOrUpdate

Indexes

Index name	Columns	Type	Purpose
organizations_pkey	id	BTREE	Primary key lookup

- *EF Core entity: Organization.cs in AiMinutes.Domain/Entities/*
- *plan column: .HasConversion() in OnModelCreating — EF stores enum as integer*
- *No rows are ever hard-deleted from organizations — soft-delete via is_active flag if needed in future*

EF Core entity

```
public class Organization
{
    public Guid     Id                    { get; set; }
    public string   Name                  { get; set; } = string.Empty;
    public PlanTier Plan                  { get; set; } = PlanTier.Free;
    public int      MeetingsUsedThisMonth { get; set; } = 0;
    public DateTime PlanRenewsAt          { get; set; }
    public DateTime CreatedAt             { get; set; }
    public DateTime UpdatedAt             { get; set; }

    // Navigation properties
    public ICollection<AppUser>        Users          { get; set; } = [];
    public ICollection<Meeting>        Meetings       { get; set; } = [];
    public ICollection<EmailRecipient> EmailRecipients{ get; set; } = [];
}
```

users — Application users — extends ASP.NET Identity

Extends IdentityUser from ASP.NET Core Identity. The Identity framework manages password hashing, lockout, and claims automatically. We add org_id and role to link each user to their organization.

Column	Type	Null	Default	Description / Constraint
id	UUID	✗	gen_random_uuid()	Primary key — inherits from IdentityUser
org_id	UUID	✗	—	FK → organizations.id | Every user belongs to exactly one organization. Cascade delete: if org deleted, users deleted too
full_name	VARCHAR(100)	✗	—	User's display name shown in the UI and in minutes (ApprovedByName etc.)
email	VARCHAR(256)	✗	—	UNIQUE | Login email — unique platform-wide. Managed by ASP.NET Identity (NormalizedEmail index)
role	VARCHAR(20)	✗	'Member'	Admin | OrgOwner | Member — also stored in JWT claims and AspNetUserRoles
password_hash	TEXT	✓	NULL	BCrypt hash managed by ASP.NET Identity — never access directly, use UserManager
created_at	TIMESTAMPTZ	✗	NOW()	Account creation timestamp
is_active	BOOLEAN	✗	TRUE	Soft delete flag. Set to FALSE instead of deleting. Checked on login.
email_confirmed	BOOLEAN	✗	FALSE	Managed by Identity — TRUE once email verified (future feature)
lockout_end	TIMESTAMPTZ	✓	NULL	Identity account lockout — auto-set after failed login attempts
access_failed_count	INT	✗	0	Identity failed login counter — resets on successful login

Indexes

Index name	Columns	Type	Purpose
users_pkey	id	BTREE	Primary key
users_org_id_idx	org_id	BTREE	Fast lookup of all users in an org
AspNetUsers_NormalizedEmail	NormalizedEmail	BTREE	Identity login by email — created automatically by Identity

- *C# class: public class AppUser : IdentityUser — add only org_id, full_name, role, is_active, created_at*
- *Use _userManager.CreateAsync(user, password) — never set password_hash directly*
- *Soft delete only: set is_active = false. Never hard-delete users — meetings reference them*

EF Core entity

```
public class AppUser : IdentityUser<Guid>
{
```

```
    public Guid     OrgId     { get; set; }
    public string   FullName  { get; set; } = string.Empty;
    public string   Role      { get; set; } = "Member";
    public bool     IsActive  { get; set; } = true;
    public DateTime CreatedAt { get; set; }

    // Navigation
    public Organization Organization { get; set; } = null!;
}
```

meetings — Core entity — one row per meeting session

The central table. Every other table (transcripts, minutes, email_delivery_log) hangs off a meeting. The status column is the most important field — the frontend polls it every 5 seconds to drive the UI state machine.

Column	Type	Null	Default	Description / Constraint
id	UUID	✗	gen_random_uuid()	Primary key
org_id	UUID	✗	—	FK → organizations.id | Multi-tenant isolation. Always filter by org_id — never return meetings from other orgs
created_by	UUID	✗	—	FK → users.id | User who created this meeting. Used for CreatedByName in API responses
approved_by	UUID	✓	NULL	FK → users.id | User who approved the minutes. NULL until approved. Used for ApprovedByName
title	VARCHAR(255)	✗	—	Meeting title set by the user (3–255 chars)
description	TEXT	✓	NULL	Optional meeting context (max 500 chars)
status	VARCHAR(30)	✗	'created'	Pipeline state machine — see MeetingStatus enum. Values: created | uploading | transcribing | generating_minutes | pending_validation | approved | sent | failed
audio_path	TEXT	✓	NULL	Relative path to audio file on disk or blob storage URL. NULL until uploaded
audio_duration_secs	INT	✓	NULL	Duration of the audio file in seconds — stored for display in UI
audio_mime_type	VARCHAR(50)	✓	NULL	MIME type of uploaded audio e.g. audio/mpeg, audio/webm
scheduled_at	TIMESTAMPTZ	✓	NULL	When the meeting occurred. Defaults to created_at if not set by user
ended_at	TIMESTAMPTZ	✓	NULL	Set when audio upload completes and background job is enqueued
approved_at	TIMESTAMPTZ	✓	NULL	Set when user approves minutes. NULL until approved
created_at	TIMESTAMPTZ	✗	NOW()	Record creation timestamp (UTC)
updated_at	TIMESTAMPTZ	✗	NOW()	Last modified timestamp — updated on every status change
is_deleted	BOOLEAN	✗	FALSE	Soft delete. Filter with .Where(m => !m.IsDeleted) in all queries via EF global query filter

Indexes

Index name	Columns	Type	Purpose
meetings_pkey	id	BTREE	Primary key
meetings_org_id_idx	org_id	BTREE	List all meetings for an org — most common query
meetings_status_idx	status	BTREE	Filter by status (e.g. all pending_validation meetings)
meetings_org_status_idx	org_id, status	BTREE	Composite — org dashboard filtered by status
meetings_created_at_idx	created_at DESC	BTREE	Default sort — newest first

- *Global query filter in AppDbContext: modelBuilder.Entity().HasQueryFilter(m => !m.IsDeleted)*
- *approved_by is SET NULL on user soft-delete (not cascaded) — keeps meeting history intact*
- *Status transitions are one-way: never go backwards. Validate transition in MeetingService before saving*

EF Core entity

```
public class Meeting
{
    public Guid          Id               { get; set; }
    public Guid          OrgId            { get; set; }
    public Guid          CreatedBy        { get; set; }
    public Guid?         ApprovedBy       { get; set; }
    public string        Title            { get; set; } = string.Empty;
    public string?       Description      { get; set; }
    public MeetingStatus Status           { get; set; } = MeetingStatus.Created;
    public string?       AudioPath        { get; set; }
    public int?          AudioDurationSecs{ get; set; }
    public string?       AudioMimeType    { get; set; }
    public DateTime?     ScheduledAt      { get; set; }
    public DateTime?     EndedAt          { get; set; }
    public DateTime?     ApprovedAt       { get; set; }
    public DateTime      CreatedAt        { get; set; }
    public DateTime      UpdatedAt        { get; set; }
    public bool          IsDeleted        { get; set; } = false;

    // Navigation
    public Organization  Organization     { get; set; } = null!;
    public AppUser       Creator          { get; set; } = null!;
    public Transcript?   Transcript       { get; set; }
    public Minutes?      Minutes          { get; set; }
}
```

transcripts — Whisper ASR output — one row per meeting

Created by the background job after Whisper processes the audio. One-to-one with meetings. The raw_text feeds the LLM for minutes generation. The segments JSONB stores speaker-diarised output for display in the UI.

Column	Type	Null	Default	Description / Constraint
■ id	*UUID*	✗	gen_random_uuid()	Primary key
■ meeting_id	*UUID*	✗	—	FK → meetings.id | One-to-one with meetings. UNIQUE constraint ensures only one transcript per meeting
raw_text	*TEXT*	✗	—	Full plain-text transcript — all speakers combined in order. Fed to LLM.
segments	*JSONB*	✓	NULL	Array of speaker segments from Whisper: [{speaker, text, start_ms, end_ms}]. NULL if Whisper did not return diarisation data
whisper_model	*VARCHAR(30)*	✓	NULL	Whisper model used e.g. whisper-1. Stored for debugging and cost tracking
duration_secs	*INT*	✓	NULL	Audio duration as reported by Whisper (cross-check with meetings.audio_duration_secs)
word_count	*INT*	✓	NULL	Approximate word count of raw_text — useful for estimating LLM token cost
created_at	*TIMESTAMPTZ*	✗	NOW()	When transcription completed

Indexes

Index name	Columns	Type	Purpose
transcripts_pkey	id	BTREE	Primary key
transcripts_meeting_uq	meeting_id	BTREE UNIQUE	Enforces one transcript per meeting

- *JSONB segments example: [{"speaker":"Speaker 1","text":"Good morning","start_ms":0,"end_ms":1200}]*
- *raw_text is never shown directly to users — only the structured minutes from the minutes table are shown*
- *Retain transcripts even after minutes are generated — useful for re-generating minutes with a different prompt in V2*

EF Core entity

```
public class Transcript
{
    public Guid    Id           { get; set; }
    public Guid    MeetingId    { get; set; }
    public string  RawText      { get; set; } = string.Empty;
    public string? Segments     { get; set; }   // JSON string
    public string? WhisperModel { get; set; }
    public int?    DurationSecs { get; set; }
    public int?    WordCount    { get; set; }
    public DateTime CreatedAt   { get; set; }
```

```
    public Meeting Meeting      { get; set; } = null!;
}
```

minutes — AI-generated draft + approved minutes — one row per meeting

The most important table for the user. Created by the background job after LLM processing. draft_json holds the raw AI output. approved_html holds the final human-validated content. search_vector enables full-text search across all approved minutes.

Column	Type	Null	Default	Description / Constraint
■ id	UUID	✗	gen_random_uuid()	Primary key
■ meeting_id	UUID	✗	—	FK → meetings.id | One-to-one with meetings. UNIQUE constraint ensures one minutes record per meeting
status	VARCHAR(25)	✗	'pending_validation'	pending_validation | approved | sent. Mirrors meeting.status for minutes-specific queries
version	INT	✗	1	Incremented on every PUT /api/meetings/{id}/minutes save. Starts at 1
draft_json	JSONB	✗	—	Raw LLM output — MinutesDraftDto shape. Displayed in TipTap editor for validation
approved_html	TEXT	✓	NULL	Final HTML from TipTap after user approves. Sanitised before storage. NULL until approved
■ approved_by	UUID	✓	NULL	FK → users.id | User who approved. NULL until approved. SET NULL if user is deactivated
approved_at	TIMESTAMPTZ	✓	NULL	Approval timestamp (UTC). NULL until approved
llm_model	VARCHAR(50)	✓	NULL	LLM model used e.g. gpt-4o. Stored for debugging and cost tracking
prompt_tokens	INT	✓	NULL	Tokens used in the prompt sent to LLM — for cost monitoring
completion_tokens	INT	✓	NULL	Tokens returned by LLM — for cost monitoring
search_vector	TSVECTOR	✓	NULL	Auto-generated from approved_html. Used by GIN index for full-text search. NULL until approved
created_at	TIMESTAMPTZ	✗	NOW()	When LLM generated the draft minutes
updated_at	TIMESTAMPTZ	✗	NOW()	Last edited by user in validation editor

Indexes

Index name	Columns	Type	Purpose
minutes_pkey	id	BTREE	Primary key
minutes_meeting_uq	meeting_id	BTREE UNIQUE	One minutes record per meeting
minutes_search_gin	search_vector	GIN	Full-text search — required for /api/minutes/search

minutes_status_idx	status	BTREE	Filter by pending_validation for dashboard

- *search_vector is auto-updated by PostgreSQL generated column: GENERATED ALWAYS AS (to_tsvector('english', coalesce(approved_html,''))) STORED*
- *Configure in EF Core: b.HasGeneratedTsVectorColumn(m => m.SearchVector, "english", m => new { m.ApprovedHtml }).HasIndex(m => m.SearchVector).HasMethod("GIN")*
- *draft_json shape matches MinutesDraftDto exactly — never store partial objects*
- *approved_html must be sanitised with HtmlSanitizer NuGet package before INSERT/UPDATE*

EF Core entity

```
public class Minutes
{
    public Guid         Id               { get; set; }
    public Guid         MeetingId        { get; set; }
    public string       Status           { get; set; } = "pending_validation";
    public int          Version          { get; set; } = 1;
    public string       DraftJson        { get; set; } = string.Empty;  // JSON
    public string?      ApprovedHtml     { get; set; }
    public Guid?        ApprovedBy       { get; set; }
    public DateTime?    ApprovedAt       { get; set; }
    public string?      LlmModel         { get; set; }
    public int?         PromptTokens     { get; set; }
    public int?         CompletionTokens { get; set; }
    public NpgsqlTsVector? SearchVector  { get; set; }  // auto-generated
    public DateTime     CreatedAt        { get; set; }
    public DateTime     UpdatedAt        { get; set; }

    public Meeting      Meeting          { get; set; } = null!;
    public AppUser?     Approver         { get; set; }
    public ICollection<MinutesVersion> Versions { get; set; } = [];
}
```

minutes_versions — Edit history — one row per save in the validation editor

Every time a user saves changes in the TipTap editor, the current draft_json is snapshotted here before overwriting. This gives you a full edit history and allows diff views between versions.

Column	Type	Null	Default	Description / Constraint
■ id	UUID	✗	gen_random_uuid()	Primary key
■ minutes_id	UUID	✗	—	FK → minutes.id | Minutes record this version belongs to. Cascade delete — versions go when minutes go
■ edited_by	UUID	✗	—	FK → users.id | User who made this edit. SET NULL on user deactivation
version_number	INT	✗	—	Snapshot of minutes.version at the time of this save
draft_json	JSONB	✗	—	Full MinutesDraftDto snapshot at this version — the state BEFORE the edit was applied
edit_summary	VARCHAR(200)	✓	NULL	Optional human note about what was changed (from UpdateMinutesDto.EditSummary)
created_at	TIMESTAMPTZ	✗	NOW()	When this version was saved

Indexes

Index name	Columns	Type	Purpose
versions_pkey	id	BTREE	Primary key
versions_minutes_id_idx	minutes_id, version_number DESC	BTREE	Get version history for a minutes record newest first

■ *Insert a version snapshot BEFORE updating minutes.draft_json — not after*
■ *Only keep last 20 versions per meeting — add cleanup job or BEFORE INSERT trigger if needed*

EF Core entity

```
public class MinutesVersion
{
    public Guid     Id            { get; set; }
    public Guid     MinutesId     { get; set; }
    public Guid     EditedBy      { get; set; }
    public int      VersionNumber { get; set; }
    public string   DraftJson     { get; set; } = string.Empty;
    public string?  EditSummary   { get; set; }
    public DateTime CreatedAt     { get; set; }

    public Minutes  Minutes       { get; set; } = null!;
    public AppUser  Editor        { get; set; } = null!;
}
```

email_recipients — Per-org email distribution list for minutes

Every approved meeting's minutes are emailed to all rows in this table that match the org_id. Managed by OrgOwner via PUT /api/org/recipients. Recipients do not need to be registered users.

Column	Type	Null	Default	Description / Constraint
id	UUID	✗	gen_random_uuid()	Primary key
org_id	UUID	✗	—	FK → organizations.id | The organization this recipient belongs to. Cascade delete.
email	VARCHAR(255)	✗	—	Recipient email address — validated as proper email format on insert
name	VARCHAR(255)	✗	—	Display name shown in SendGrid 'To:' field e.g. 'Jane Smith'
is_active	BOOLEAN	✗	TRUE	Set to FALSE when SendGrid reports a hard bounce — do not auto-delete
bounce_reason	TEXT	✓	NULL	Populated from SendGrid bounce webhook event reason field
created_at	TIMESTAMPTZ	✗	NOW()	When this recipient was added

Indexes

Index name	Columns	Type	Purpose
recipients_pkey	id	BTREE	Primary key
recipients_org_id_idx	org_id, is_active	BTREE	Get active recipients for an org (most common query)

- *Unique constraint on (org_id, email) — prevent duplicate recipients per org*
- *When SendGrid reports a hard bounce: set is_active=FALSE and store bounce_reason — do not silently delete*
- *PUT /api/org/recipients replaces the entire list — use a transaction: delete all for org, then insert new list*

EF Core entity

```
public class EmailRecipient
{
    public Guid     Id           { get; set; }
    public Guid     OrgId        { get; set; }
    public string   Email        { get; set; } = string.Empty;
    public string   Name         { get; set; } = string.Empty;
    public bool     IsActive     { get; set; } = true;
    public string?  BounceReason { get; set; }
    public DateTime CreatedAt    { get; set; }

    public Organization Organization { get; set; } = null!;
}
```

email_delivery_log — SendGrid delivery event log — one row per event per recipient

Populated by the POST /webhooks/sendgrid handler. Lets you show 'Delivered to 5 of 6 recipients, 1 bounced' on the meeting detail page. One row per SendGrid event — a single email send creates multiple rows (one per recipient per event type).

Column	Type	Null	Default	Description / Constraint
■ id	*UUID*	✗	gen_random_uuid()	Primary key
■ meeting_id	*UUID*	✗	—	FK → meetings.id | The meeting whose minutes were emailed. Matched via sendgrid_message_id
sendgrid_message_id	*TEXT*	✗	—	sg_message_id from SendGrid webhook — used to look up meeting on webhook receipt
recipient_email	*VARCHAR(255)*	✗	—	The individual recipient this event is for
event_type	*VARCHAR(20)*	✗	—	delivered | bounce | open | click | spamreport — from SendGrid event.event field
event_at	*TIMESTAMPTZ*	✗	—	When the event occurred — converted from SendGrid's Unix timestamp
bounce_reason	*TEXT*	✓	NULL	Only populated on bounce events — raw reason string from SendGrid
raw_payload	*JSONB*	✓	NULL	Full raw SendGrid event JSON — stored for debugging. Can be purged after 90 days
created_at	*TIMESTAMPTZ*	✗	NOW()	When the webhook was received and this row was inserted

Indexes

Index name	Columns	Type	Purpose
delivery_pkey	id	BTREE	Primary key
delivery_meeting_idx	meeting_id	BTREE	Get all delivery events for a meeting
delivery_sg_msg_id_idx	sendgrid_message_id	BTREE	Match incoming webhook to a meeting — critical for webhook handler speed
delivery_event_type_idx	event_type	BTREE	Count delivered vs bounced per meeting

- *Idempotent insert: use INSERT ... ON CONFLICT DO NOTHING to handle SendGrid retrying the same event*
- *raw_payload can be large — add a scheduled job to NULL it out after 90 days to save storage*
- *Join with email_recipients on recipient_email to show delivery status per named recipient*

EF Core entity

```
public class EmailDeliveryLog
{
    public Guid     Id                { get; set; }
    public Guid     MeetingId         { get; set; }
    public string   SendGridMessageId { get; set; } = string.Empty;
    public string   RecipientEmail    { get; set; } = string.Empty;
```

```
    public string   EventType          { get; set; } = string.Empty;
    public DateTime EventAt            { get; set; }
    public string?  BounceReason       { get; set; }
    public string?  RawPayload         { get; set; }
    public DateTime CreatedAt          { get; set; }

    public Meeting  Meeting            { get; set; } = null!;
}
```

ASP.NET Core Identity Tables

These four tables are created automatically by EF Core when you configure IdentityDbContext. You do not write migrations for them manually — dotnet ef migrations add handles them. They work alongside your custom tables.

Table	Purpose	Key columns
`AspNetUsers`	Stores all users — maps 1:1 with your AppUser class	Id (UUID PK), Email, NormalizedEmail (indexed), PasswordHash
`AspNetRoles`	Role definitions — Admin, OrgOwner, Member	Id (UUID PK), Name, NormalizedName (indexed)
`AspNetUserRoles`	Junction table linking users to roles (many-to-many)	UserId (FK → AspNetUsers), RoleId (FK → AspNetRoles). Comp
`AspNetUserClaims`	Per-user claims stored in the DB (org_id, org_role)	Id (PK), UserId (FK → AspNetUsers), ClaimType, ClaimValue

All Table Relationships

From	Cardinality	To	Via column	Note
organizations	1 : MANY	users	users.org_id	Cascade delete
organizations	1 : MANY	meetings	meetings.org_id	Cascade delete
organizations	1 : MANY	email_recipients	email_recipients.org_id	Cascade delete
users	1 : MANY	meetings	meetings.created_by	Restrict delete — deactivate instead
users	1 : MANY	minutes	minutes.approved_by	SET NULL on deactivate
users	1 : MANY	minutes_versions	minutes_versions.edited_by	SET NULL on deactivate
meetings	1 : 1	transcripts	transcripts.meeting_id	Cascade delete
meetings	1 : 1	minutes	minutes.meeting_id	Cascade delete
meetings	1 : MANY	email_delivery_log	email_delivery_log.meeting_id	Cascade delete
minutes	1 : MANY	minutes_versions	minutes_versions.minutes_id	Cascade delete
AspNetUsers	1 : 1	users	users.id = AspNetUsers.Id	Same UUID — AppUser extends IdentityUser
AspNetUsers	MANY : MANY	AspNetRoles	via AspNetUserRoles	Managed by Identity

EF Core Setup & Migration Commands

AppDbContext — OnModelCreating key configurations

```
protected override void OnModelCreating(ModelBuilder b)
{
    base.OnModelCreating(b);   // Must call — sets up Identity tables

    // ■■ organizations ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<Organization>()
        .Property(o => o.Plan).HasConversion<int>();   // enum as int

    // ■■ meetings ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<Meeting>()
        .HasQueryFilter(m => !m.IsDeleted)             // global soft delete
        .HasIndex(m => m.OrgId)
        .HasIndex(m => new { m.OrgId, m.Status });

    b.Entity<Meeting>()
        .Property(m => m.Status)
        .HasConversion<string>();                       // enum as string

    // ■■ minutes — full-text search ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<Minutes>()
        .HasGeneratedTsVectorColumn(
            m => m.SearchVector,
            "english",
            m => new { m.ApprovedHtml })
        .HasIndex(m => m.SearchVector)
        .HasMethod("GIN");

    // ■■ transcripts ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<Transcript>()
        .HasIndex(t => t.MeetingId).IsUnique();        // one per meeting

    // ■■ minutes_versions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<MinutesVersion>()
        .HasIndex(v => new { v.MinutesId, v.VersionNumber });

    // ■■ email_recipients — unique per org ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<EmailRecipient>()
        .HasIndex(r => new { r.OrgId, r.Email }).IsUnique();

    // ■■ email_delivery_log ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    b.Entity<EmailDeliveryLog>()
        .HasIndex(d => d.SendGridMessageId)
        .HasIndex(d => d.MeetingId);
}
```

Migration CLI commands

```
# Run from solution root
cd src/AiMinutes.API

# First migration — creates ALL tables (Identity + your custom tables)
dotnet ef migrations add InitialCreate \
    --project ../AiMinutes.Infrastructure \
    --startup-project .
```

```
# Apply to database
dotnet ef database update \
    --project ../AiMinutes.Infrastructure \
    --startup-project .

# After any entity change — create a new migration
dotnet ef migrations add AddBounceReasonToRecipients \
    --project ../AiMinutes.Infrastructure \
    --startup-project .

# View the generated SQL before applying (good practice)
dotnet ef migrations script \
    --project ../AiMinutes.Infrastructure \
    --startup-project .

# Roll back the last migration (development only)
dotnet ef migrations remove \
    --project ../AiMinutes.Infrastructure \
    --startup-project .
```

Enable PostgreSQL extensions (run once on new DB)

```
-- Run in psql or via a migration
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";   -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS "pg_trgm";     -- fuzzy text search
CREATE EXTENSION IF NOT EXISTS "unaccent";    -- accent-insensitive search
```